Milestone #2

CEPARCO Group 3

ALCASID, AIZY
BILANES, KENDRICK
CORPUZ, GERALD
ROA, LUIS

Problem: Local Alignment

Smith-Waterman Algorithm Parallelization

```
Score matrix H (5 x 6):

          0   1   2   3   4   5
              A   T   G   C   T
  0       0   0   0   0   0   0
  1 A     0   1   0   0   0   0
  2 G     0   0   0   1   0   0
  3 C     0   0   0   0   2   0
  4 T     0   0   1   0   0   3

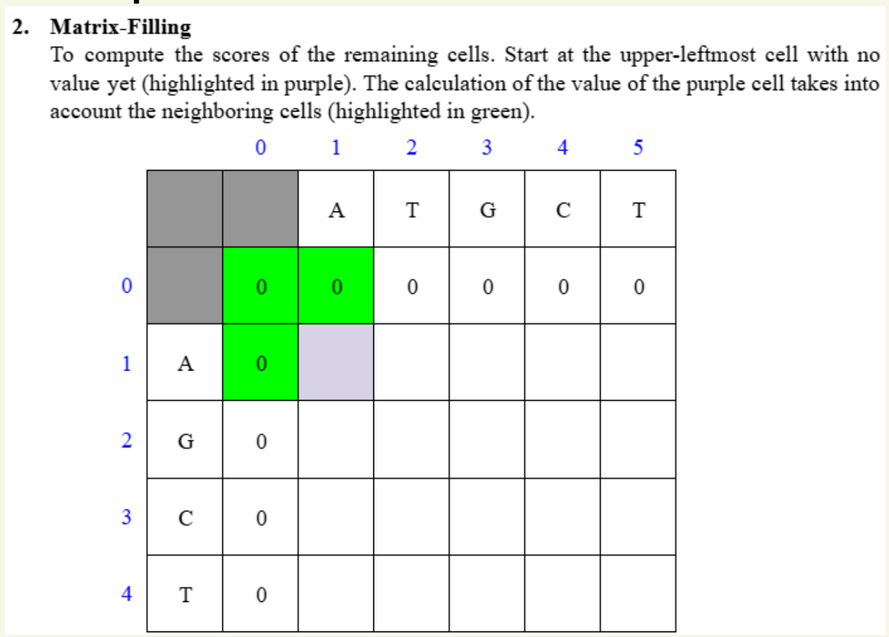
Max score: 3
Max cell : H(4, 5)

Alignment: length 3 (3 matches, 0 mismatches, 0 gaps)
Range    : a[2..4], b[3..5]

  a       2  GCT
             |||
  b       3  GCT
```

Problem

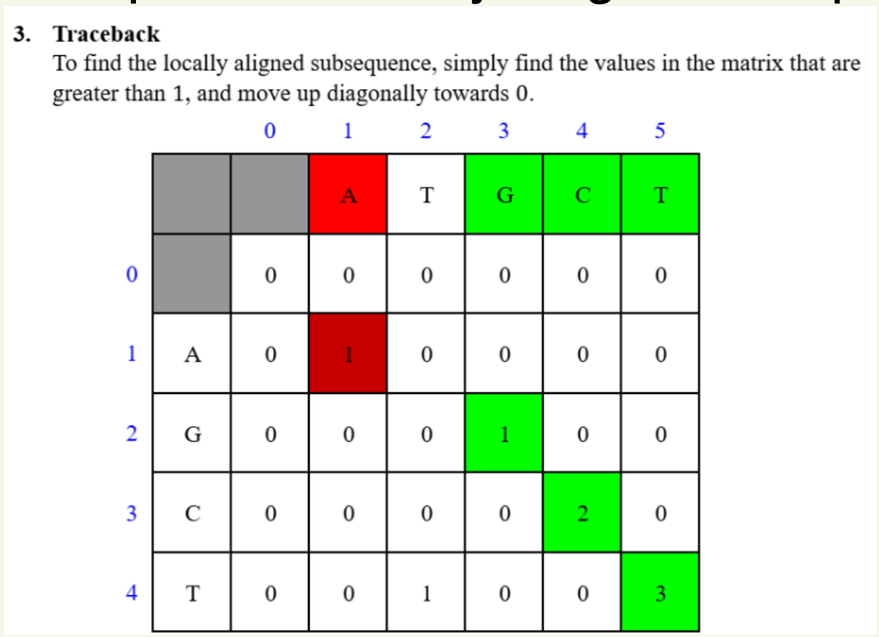
- Main bottleneck for sequential processing comes from the Matrix Filling (roughly M x N operations) of the Smith-Waterman algorithm; the execution time for traceback (at most M + N operations) is negligible since it only follows a path from the best cell to find the optimal locally aligned sequence.



2. Matrix-Filling

To compute the scores of the remaining cells. Start at the upper-leftmost cell with no value yet (highlighted in purple). The calculation of the value of the purple cell takes into account the neighboring cells (highlighted in green).

			0	1	2	3	4	5
				A	T	G	C	T
0		0	0	0	0	0	0	0
1	A	0						
2	G	0						
3	C	0						
4	T	0						



3. Traceback

To find the locally aligned subsequence, simply find the values in the matrix that are greater than 1, and move up diagonally towards 0.

			0	1	2	3	4	5
				A	T	G	C	T
0		0	0	0	0	0	0	0
1	A	0	1	0	0	0	0	0
2	G	0	0	0	1	0	0	0
3	C	0	0	0	0	2	0	0
4	T	0	0	1	0	0	3	0

Progress

- week 7 - setup Github and Colab notebook, decided on scoring parameters and test sequences.
- week 8 - Implement partial Sequential C version for Smith-Waterman algorithm
- week 9 - Finish Sequential C version for Smith-Waterman algorithm with traceback. Record the C baseline execution time for future comparison with CUDA version.
- week 10 - Start CUDA version for Smith-Waterman Matrix Filling. Present Mid-Project Demo

Approach

- Python version of the Smith-Waterman algorithm serves as the reference for the correctness of the sequential C version. Correctness of the Python version is verified by handwork.

```
/content/sw_repo/smith-waterman
c  colab  data  README.md  results  scripts
Test sequences (pure-Python reference, match +1 / mismatch -1 / gap -2):
  ok   test_paper.fasta          4 x 5   score=3   end=(4, 5) GCT
  ok   test_identical.fasta      8 x 8   score=8   end=(8, 8) ACGTACGT
  ok   test_nomatch.fasta        8 x 8   score=0   end=(0, 0) (no alignment)
  ok   test_gap.fasta           12 x 13  score=10  end=(12, 13) ACGTCA-TGCATG

Benchmark sequences (format check only; scored by the C baseline in Week 9):
  ok   bench_1000.fasta       1000 x 1000, alphabet ACGT
  ok   bench_5000.fasta       5000 x 5000, alphabet ACGT
  ok   bench_10000.fasta      10000 x 10000, alphabet ACGT

All checks passed.
```

Approach

- Sequential C version of the Smith-Waterman algorithm serves as the baseline execution time comparison against CUDA version of the Smith-Waterman algorithm.

```
Score matrix H (5 x 6):

          0   1   2   3   4   5
              A   T   G   C   T
  0       0   0   0   0   0   0
  1 A     0   1   0   0   0   0
  2 G     0   0   0   1   0   0
  3 C     0   0   0   0   2   0
  4 T     0   0   1   0   0   3

Max score: 3
Max cell : H(4, 5)

Alignment: length 3 (3 matches, 0 mismatches, 0 gaps)
Range    : a[2..4], b[3..5]

  a      2  GCT
            |||
  b      3  GCT

Re-score : 3 (OK)
Checksum : 8 (sum of all matrix cells)
```

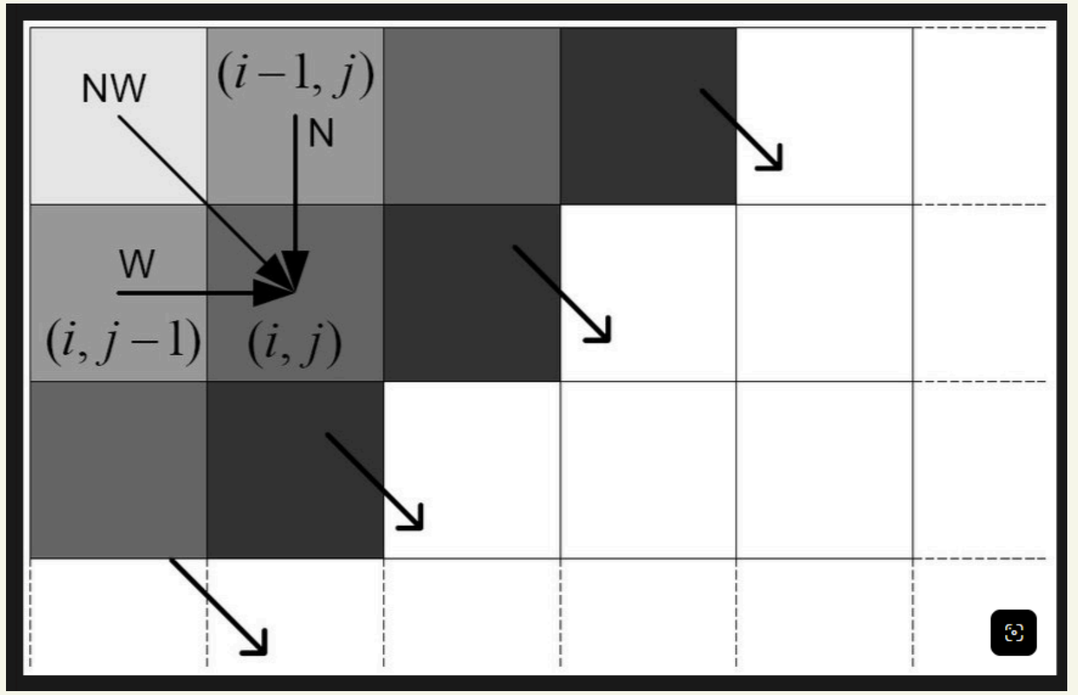
```
Smith-Waterman (sequential C)
File     : data/bench_10000.fasta
Lengths  : M = 10000 (rows), N = 10000 (columns)
Scoring  : match +1, mismatch -1, gap -2
Matrix   : 100020001 cells, 381.5 MB

Max score: 1536
Max cell : H(5403, 9807)

Alignment: length 2069 (1863 matches, 85 mismatches, 121 gaps)
Range    : a[3387..5403], b[7808..9807]
```

Approach

- CUDA Version for the Smith-Waterman algorithm is the main goal of the project. Parallelization comes from the Matrix Filling via anti-diagonals which can be split among CUDA threads.



Baseline Timing

- Ran with Google Colab T4 GPU Hardware Accelerator and Python 3 Runtime Type

Environment

- **date:** 2026-11-13 12:14
- **platform:** Linux-6.6.159+-x86_64-with-glibc2.39
- **cpu:** Intel(R) Xeon(R) CPU @ 2.00GHz
- **logical_cpus:** 2
- **gpu:** Tesla T4, 15360 MiB, 580.82.07
- **compiler:** gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
- **flags:** -std=c99 -O2 -Wall -Wextra
- **python:** 3.13.16

Baseline Timing

- Baseline Timing of the full sequential C version for the Smith-Waterman algorithm, Traceback is a very small operation compared to Matrix Filling and is neglible shown through the values.

Input	Runs	Fill median	Fill min-max	Traceback median	Alloc median	Total median
bench_1000	5	0.0020	0.0020 - 0.0021	0.000003	0.0001	0.0021
bench_5000	5	0.0495	0.0485 - 0.0512	0.000049	0.0464	0.0960
bench_10000	3	0.2111	0.2072 - 0.2159	0.000092	0.1925	0.4086

** bench_1000 - input sequences = 1000 characters in length*
bench_5000 - input sequences = 5000 characters in length
bench_10000 - input sequences = 10000 characters in length

Baseline Timing

- Baseline Timing of the full sequential C version for the Smith-Waterman algorithm without the Traceback to prove it is negligible.

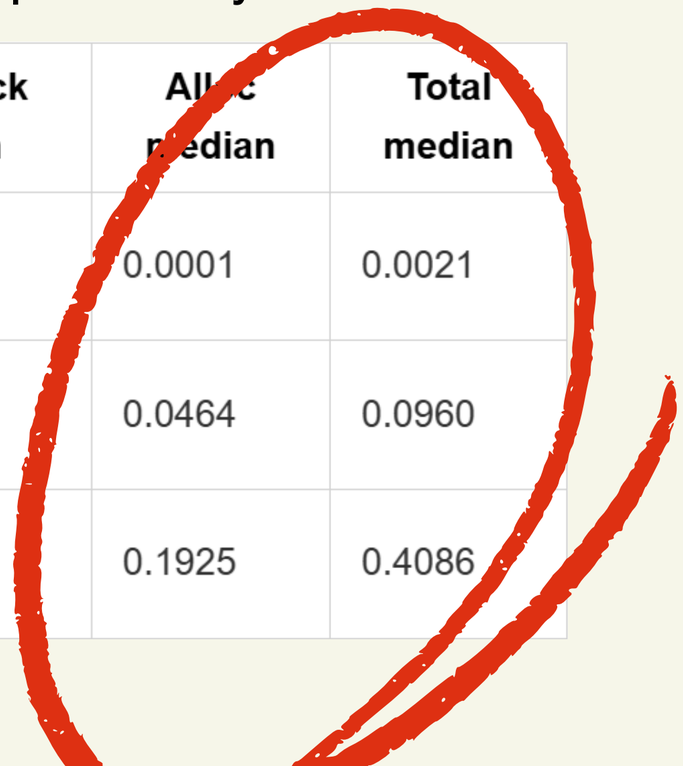
Input	Runs	Fill median	Total median
bench_1000	5	0.0019	0.0021
bench_5000	5	0.0508	0.0998
bench_10000	3	0.1980	0.3859

** bench_1000 - input sequences = 1000 characters in length*
bench_5000 - input sequences = 5000 characters in length
bench_10000 - input sequences = 10000 characters in length

Baseline Timing

- Allocation Timings: Allocation is setting up the matrix itself and zero-initializing it. bench_1000's matrix is about 4 MB; bench_5000 and bench_10000 are about 95 MB and 381 MB respectively.

Input	Runs	Fill median	Fill min-max	Traceback median	Alloc median	Total median
bench_1000	5	0.0020	0.0020 - 0.0021	0.000003	0.0001	0.0021
bench_5000	5	0.0495	0.0485 - 0.0512	0.000049	0.0464	0.0960
bench_10000	3	0.2111	0.2072 - 0.2159	0.000092	0.1925	0.4086



** bench_1000 - input sequences = 1000 characters in length*
bench_5000 - input sequences = 5000 characters in length
bench_10000 - input sequences = 10000 characters in length

Baseline Timing

- Allocation Timings: Google Colab's CPU at max has a cache size of 38.5 MB, which does not fit the allocation for bench_5000 and bench_10000.

```
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 46 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          2
  On-line CPU(s) list:           0,1
Vendor ID:                       GenuineIntel
  Model name:                    Intel(R) Xeon(R) CPU @ 2.00GHz
    CPU family:                  6
    Model:                       85
    Thread(s) per core:          2
    Core(s) per socket:          1
    Socket(s):                   1
    Stepping:                    3
    BogoMIPS:                    4000.35
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                                 e mca cmov pat pse36 clflush mmx fxsr sse sse2 ss h
                                 t syscall nx pdpe1gb rdtscp lm constant_tsc rep_goo
                                 d nopl xtopology nonstop_tsc cpuid tsc_known_freq p
                                 ni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2ap
                                 ic movbe popcnt aes xsave avx f16c rdrand hyperviso
                                 r lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp fs
                                 gsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invp
                                 cid rtm mpx avx512f avx512dq rdseed adx smap clflus
                                 hopt clwb avx512cd avx512bw avx512vl xsaveopt xsave
                                 c xgetbv1 xsaves arat md_clear arch_capabilities
```

```
Virtualization features:
  Hypervisor vendor:             KVM
  Virtualization type:           full
Caches (sum of all):
  L1d:                           32 KiB (1 instance)
  L1i:                           32 KiB (1 instance)
  L2:                            1 MiB (1 instance)
  L3:                            38.5 MiB (1 instance)
```

CUDA Matrix Filling

- Independent CPU traceback confirms the CUDA matrix fill matches the sequential C baseline across all benchmark scales.

Benchmark File	Matrix Size	CUDA Max Score	Checksum Match
bench_1000	3.8 MB (1M cells)	160 at H(784, 995)	**1,563,453 (OK)**
bench_5000	95.4 MB (25M cells)	805 at H(2313, 1448)	**1,445,075,095 (OK)**
bench_10000	381.5 MB (100M cells)	1536 at H(5403, 9807)	**1,009,197,635 (OK)**

CUDA Timing

- A 100× workload increase only slows GPU fill by 8.4×, while VRAM allocation dominates total runtime at 10k.

Input	Alloc + Zero	Fill Median	Host Trace	Total Time
bench_1000	0.0005 s	**0.0073 s**	0.000008 s	**0.0079 s**
bench_5000	0.0474 s	**0.0327 s**	0.000048 s	**0.0802 s**
bench_10000	0.2019 s	**0.0616 s**	0.000082 s	**0.2690 s**

Thank you

Group 3

2026 NOVEMBER 11